

JavaScript - Day 3

Destructuring, Spread & Rest Operators, Numbers & Math, Template Literals

Student Notes & Interview Preparation Guide

Topics covered: Object & Array Destructuring | Spread Operator | Rest Operator | Shallow vs Deep Copy
| Numbers & Special Values | Floating Point Precision | Math Object | Template Literals

Introduction

As applications become larger, working with arrays, objects, strings, and functions efficiently becomes very important.

Today's topics help developers write:

- Cleaner code
- Shorter code
- More readable code
- Modern JavaScript code
- React-friendly code

These concepts are heavily used in:

- React Development
- API Handling
- Front-End Projects
- Node.js Applications
- Real-world Software Development

1. Destructuring

Why Do We Need Destructuring?

In JavaScript, data is usually stored inside arrays and objects. Traditionally, we access values like this:

```
const user = {  
  name: "Rahul",  
  age: 22  
};  
  
console.log(user.name);  
console.log(user.age);
```

For small objects this is okay. But in real applications:

- Objects may contain many properties
- API responses may contain huge amounts of data
- Nested objects become difficult to access

Destructuring provides a shorter and cleaner way to extract values.

```
const { name, age } = user;
```

Benefits

- Cleaner syntax
- Less repetitive code
- Better readability
- Useful in APIs and React
- Makes function parameters cleaner
- Avoids temporary variables

What is Destructuring?

Destructuring is a JavaScript feature that allows us to extract values from arrays and objects and store them into variables in a shorter way.

Without Destructuring

```
const book = {  
  title: "When Breath Becomes Air",  
  author: "Paul Kalanithi"  
};  
  
const title = book.title;  
const author = book.author;
```

With Destructuring

```
const { title, author } = book;
```

Much cleaner and easier to read.

Object Destructuring

Used to unpack object properties into variables.

```
const book = {  
  title: "When Breath Becomes Air",  
  author: "Paul Kalanithi",  
  year: 2012,  
  genre: "Autobiography"  
};  
  
const { title, author, year, genre } = book;  
  
console.log(title);  
console.log(author);
```

Output:

```
When Breath Becomes Air  
Paul Kalanithi
```

Important Rule

For objects, order does NOT matter:

```
const { year, title } = book;
```

Works perfectly.

Renaming Variables

Sometimes variable names already exist.

```
const { title: x1, author: x2 } = book;  
  
console.log(x1);  
console.log(x2);
```

Here:

```
title → x1  
author → x2
```

Real World Uses of Object Destructuring

API Responses

```
const response = {
  id: 1,
  name: "John",
  email: "john@gmail.com"
};

const { name, email } = response;
```

Extract only required fields.

React Props

```
function User({ name, age }) {
  return <h1>{name}</h1>;
}
```

Very common in React.

Module Imports

```
import { useState, useEffect } from "react";
```

Array Destructuring

Definition

Array destructuring allows us to extract values from an array and assign them to variables.

```
let days = ["Sun", "Mon", "Tue", "Wed"];

let [a, b, c, d] = days;

console.log(a);
console.log(b);
```

Output:

```
Sun
Mon
```

Important Rule

For arrays, order is VERY important:

```
let [first] = days;
```

Gets:

```
Sun
```

If order changes, values change.

Skipping Values

```
let days = ["Sun", "Mon", "Tue", "Wed"];

let [, , x, y] = days;

console.log(x);
console.log(y);
```

Output:

```
Tue
Wed
```

Real World Uses of Array Destructuring

Multiple Return Values

```
function getCoordinates() {
  return [100, 200];
}

const [x, y] = getCoordinates();
```

Swapping Variables

```
let a = 10;
let b = 20;

[a, b] = [b, a];
```

Note: Interview Favorite.

Object.entries()

```
for (const [key, value] of Object.entries(user)) {  
  console.log(key, value);  
}
```

Nested Destructuring

Sometimes data contains objects inside objects or arrays inside arrays.

Nested Object Destructuring

```
const user = {  
  name: "Yatheswar",  
  address: {  
    city: "Vizag",  
    pincode: 530008  
  }  
};
```

Extract city directly:

```
const {  
  name,  
  address: { city }  
} = user;  
  
console.log(name);  
console.log(city);
```

Output:

```
Yatheswar  
Vizag
```

Alternative Method

```
const { address } = user;  
  
const { city, pincode } = address;
```

This approach is easier for beginners.

Nested Array Destructuring

```
const arr = [1, [2, 3]];

const [x, [y, z]] = arr;

console.log(x, y, z);
```

Output:

```
1 2 3
```

Benefits of Destructuring

Cleaner Syntax

```
const { name } = user;
```

instead of

```
const name = user.name;
```

- Less Repetition
- Better Readability
- Useful in APIs
- Useful in React
- Cleaner Function Parameters
- Avoids Temporary Variables

Common Mistakes in Destructuring

Mistake 1: Confusing object and array syntax

Incorrect

```
const [name] = user;
```

Correct

```
const { name } = user;
```

Mistake 2: Object key names must match

```
const { username } = user;
```

If user doesn't have a username key, the result is:

```
undefined
```

Mistake 3: Overusing nested destructuring

Too much nesting makes code harder to read.

Interview Summary

Array Destructuring

- Uses []
- Order matters
- Position based

Object Destructuring

- Uses { }
 - Order doesn't matter
 - Property-name based
-

2. Spread & Rest Operators (...)

Understanding the Three Dots (...)

The three dots (...) introduced in ES6 have two different uses:

1. Spread Operator
2. Rest Operator

Operator	Purpose
Spread	Expands values
Rest	Collects values

The meaning depends on where they are used.

Easy Memory Trick

Spread = Expand

Rest = Collect

Or:

Spread → Open the Box

Rest → Put Things Back Into the Box

Spread Operator

What is Spread?

Spread expands elements from arrays or objects. Think of it as: "Open the box and spread everything outside."

Why Do We Need Spread?

Because arrays and objects are stored using references. Direct copying causes problems.

```
let arr1 = [1, 2, 3];
```

```
let arr2 = arr1;
```

Both point to the same memory. Changes affect both.

Shallow Copy vs Deep Copy

Before learning spread completely, we must understand copying.

Shallow Copy

A shallow copy creates a new object only at the first level, but nested objects still share references.

```
const shallowCopy = {  
  ...original  
};
```

Example

```
const original = {  
  name: "Sai Rahul",  
  address: {  
    city: "Vizianagaram"  
  }  
};  
  
const copy = { ...original };
```

Changing Primitive Value

```
copy.name = "Mike";
```

Original remains unchanged.

Changing Nested Object

```
copy.address.city = "Vizag";
```

Original also changes. Why? Because the nested object is still shared.

Memory Visualization

```
original  
|  
|---- name  
|  
|---- address -----> { city: "Vizag" }  
  
copy  
|  
|---- name  
|  
|---- address -----> Same Object
```

Characteristics

Advantages:

- Faster
- Less memory
- Easy to create

Disadvantages:

- Nested objects are shared

Deep Copy

A deep copy creates completely independent copies of nested objects and arrays.

```
const deepCopy =  
  JSON.parse(JSON.stringify(original));
```

Now nested objects are also copied.

Example

```
deepCopy.address.city = "Vizag";
```

Original object remains unchanged.

Characteristics

Advantages:

- Safer
- Independent nested objects

Disadvantages:

- More memory
- Slower for large objects

Limitation of JSON Deep Copy

Works well for:

- Objects
- Arrays
- Strings
- Numbers

May not work correctly for:

- Functions
- Dates
- Maps
- Sets

Modern alternative:

```
const deepCopy = structuredClone(original);
```

Quick Comparison

Feature	Shallow Copy	Deep Copy
Nested Objects Shared	Yes	No
Faster	Yes	No
Less Memory	Yes	No
Safer	No	Yes

Spread Operator Examples

Spread with Arrays

```
let arr = [1, 2, 3];  
let copy = [...arr];
```

Creates a new array.

Merging Arrays

```
let arr1 = [1, 2, 3];  
let arr2 = [4, 5, 6];  
  
let merged = [...arr1, ...arr2];
```

Output:

```
[1,2,3,4,5,6]
```

Spread with Objects

```
const user = {  
  name: "John"  
};  
  
const updatedUser = {  
  ...user,  
  age: 25  
};
```

Output:

```
{  
  name: "John",  
  age: 25  
}
```

Real World Uses

- React State Updates
 - API Data Manipulation
 - Copying Arrays
 - Copying Objects
 - Merging Arrays
 - Merging Objects
-

Rest Operator

What is Rest?

Rest collects multiple values into a single array. Think of it as: "Collect the remaining items."

Rest in Arrays

```
let fruits = [  
  "apple",  
  "banana",  
  "orange",  
  "mango"  
];  
  
const [first, ...rest] = fruits;
```

Output:

```
first = apple  
  
rest = [  
  banana,  
  orange,  
  mango  
]
```

Rest in Functions

```
function sum(...numbers) {  
  return numbers;  
}  
  
sum(1,2,3,4,5);
```

Output:

```
[1,2,3,4,5]
```

Real Sum Example

```
function sum(...numbers) {  
  return numbers.reduce(  
    (acc, curr) => acc + curr,  
    0  
  );  
}
```

Real World Uses

- Variable Number of Arguments
 - React Props
 - Remaining Values in Destructuring
-

Spread vs Rest

```
// Spread  
console.log(...arr);  
  
// Rest  
function test(...values) {}
```

Easy Memory Trick

```
Spread = Expand  
Rest = Collect
```

One-Line Interview Answer

Note: Rest Operator collects multiple values into a single array, whereas Spread Operator expands elements of an array or object into individual values.

Common Mistakes

- Confusing Spread and Rest
- Assuming Spread Creates a Deep Copy
- Forgetting Spread is Shallow
- Rest Parameter Must Be Last

Incorrect

```
function test(...a, b) {}
```

Correct

```
function test(a, ...b) {}
```

3. Numbers & Math

JavaScript Number Type

JavaScript has only one numeric type:

```
Number
```

Examples:

```
10  
20  
3.14  
-100
```

All are Number type.

Special Numbers

Infinity

```
console.log(100 / 0);
```

Output:

```
Infinity
```

Negative Infinity

```
console.log(-100 / 0);
```

Output:

```
-Infinity
```

Negative Zero

```
console.log(0 / -20);
```

Output:

```
-0
```

NaN (Not a Number)

```
console.log("Math" - 100);
```

Output:

```
NaN
```

Types

```
typeof Infinity  
typeof NaN
```

Output:

```
number  
number
```

Note: Even NaN is considered a Number type.

Truthy and Falsy Values

Falsy Values

JavaScript automatically treats these as false:

```
false  
0  
-0  
null  
undefined  
NaN  
""
```

Truthy Values

Everything else. Examples:

```
"hello"  
[]  
{}  
1  
100  
true
```

Interview Question

Is an Empty Array Truthy or Falsy?

```
if([]) {  
  console.log("Hello");  
}
```

Output:

Hello

Arrays are Truthy.

Floating Point Behavior

The Problem

```
console.log(0.1 + 0.2);
```

Expected:

0.3

Actual:

0.30000000000000004

Why Does This Happen?

Humans use Base 10. Computers use Base 2. Some decimal values cannot be represented exactly in binary.

Examples:

```
0.1  
0.2  
1/3
```

JavaScript stores the nearest approximation.

IEEE 754

JavaScript stores numbers using:

```
IEEE 754 Double Precision Floating Point
```

This is why small precision errors occur.

Never Compare Floating Values Directly

Incorrect

```
0.1 + 0.2 === 0.3
```

Returns:

```
false
```

Better Approach

```
const epsilon = 0.000000000001;

if (
  Math.abs(
    (0.1 + 0.2) - 0.3
  ) < epsilon
) {
  console.log("Almost Equal");
}
```

Real World Importance

Used in:

- Banking Software
 - Scientific Applications
 - Stock Market Systems
 - Financial Calculations
-

Math Object

JavaScript provides a built-in Math object.

Math.round()

Rounds normally.

```
Math.round(4.4)
```

Output:

```
4
```

```
Math.round(4.6)
```

Output:

```
5
```

Math.floor()

Always the lower value.

```
Math.floor(4.9)
```

Output:

```
4
```

Math.ceil()

Always the upper value.

```
Math.ceil(4.1)
```

Output:

```
5
```

Math.trunc()

Removes the decimal part.

```
Math.trunc(4.9)
```

Output:

```
4
```

Difference Between floor and trunc

Positive Numbers:

```
Math.floor(4.9)
Math.trunc(4.9)
```

Both:

```
4
```

Negative Numbers:

```
Math.floor(-3.2)
```

Output:

```
-4
```

```
Math.trunc(-3.2)
```

Output:

```
-3
```

Note: Very common interview question.

Math.max()

```
Math.max(10,20,30)
```

Output:

```
30
```

Math.min()

```
Math.min(10,20,30)
```

Output:

```
10
```

Math.pow()

```
Math.pow(2,3)
```

Output:

```
8
```

Modern syntax:

```
2 ** 3
```

Math.PI

```
Math.PI
```

Output:

```
3.141592653589793
```

4. Template Literals

What are Template Literals?

Template literals are strings written using backticks.

```
` `
```

They allow:

- Variable insertion
 - Expressions
 - Multi-line strings
 - Better formatting
-

Why Were Template Literals Introduced?

Old approach:

```
let name = "Hemanth";

let msg =
  "Hello " + name;
```

Difficult to read in large strings.

Variable Interpolation

```
let name = "Hemanth";
let age = 20;

let message =
  `Hi all. My name is ${name} and my age is ${age}`;
```

Output:

```
Hi all. My name is Hemanth and my age is 20
```

Expressions Inside Template Literals

```
let a = 10;
let b = 20;

console.log(
  `Sum = ${a + b}`
);
```

Output:

```
Sum = 30
```

Functions Inside Template Literals

```
function square(n) {
  return n * n;
}

console.log(
  `Square = ${square(5)}`
);
```

Output:

```
Square = 25
```

Multi-Line Strings

```
let content = `
Line 1
Line 2
Line 3
`;
```

No need for `\n`.

Conditional Statements

```
const isLoggedIn = true;

const message =
`User is ${
  isLoggedIn
  ? "Online"
  : "Offline"
}`;
```

Output:

```
User is Online
```

Nested Template Literals

```
let username = "Uday";

let skills = [
  "HTML",
  "CSS",
  "JS"
];

const output = `
Name: ${username}
Skills: ${skills.join(", ")}
`;
```

Using Template Literals in DOM

Very common in Front-End Development.

```
const card = `
<div class="card">
  <h2>${user.name}</h2>
  <p>${user.age}</p>
</div>
`;

document.getElementById("user").innerHTML = card;
```

Real World Uses of Template Literals

Building HTML

```
const card = `

# ${title}</h1></div>`;


```

- API Data Display
- Dynamic Messages
- Email Templates
- React JSX Expressions

Common Mistakes

Using Quotes Instead of Backticks

Incorrect

```
"Hello ${name}"
```

Output:

```
Hello ${name}
```

Forgetting \${}

Incorrect

```
`Hello name`
```

Complex Logic Inside Templates

Avoid:

```
`${a > b ? someHugeCalculation() : anotherHugeCalculation()}`
```

Keep templates clean.

Interview Questions

3. What is Destructuring?
4. Difference between Object and Array Destructuring?
5. What is the Spread Operator?

6. Difference between Spread and Rest Operator?
 7. What is Shallow Copy?
 8. What is Deep Copy?
 9. Why does $0.1 + 0.2$ not equal 0.3 ?
 10. What is IEEE 754?
 11. Difference between `Math.floor()` and `Math.trunc()`?
 12. What are Template Literals?
 13. Why are Template Literals better than String Concatenation?
 14. What are Truthy and Falsy Values?
-

Practice Exercises

Exercise 1

Create a student object and extract:

- name
- age
- course

using destructuring.

Exercise 2

Merge two arrays using the spread operator.

```
[1,2,3]
[4,5,6]
```

Expected:

```
[1,2,3,4,5,6]
```

Exercise 3

Create a function using the rest operator that returns the sum of all numbers.

Exercise 4

Find:

```
Math.floor(-7.9)
Math.trunc(-7.9)
```

Explain the difference.

Exercise 5

Create a template literal that prints:

```
Hello, my name is John and I am 22 years old.
```

Mini Assignment

Create a "Student Profile Card" application.

Requirements:

15. Create a student object.
16. Use object destructuring.
17. Use template literals to build HTML.
18. Use the spread operator to copy the object.
19. Add a skills array and merge another skills array.
20. Display everything inside the DOM.

Key Takeaways

- Destructuring extracts values from arrays and objects easily.
- Object destructuring depends on key names, not order.
- Array destructuring depends on position/order.
- Spread expands values.
- Rest collects values.
- Spread creates only a shallow copy.
- Deep copy creates fully independent data.
- JavaScript uses IEEE 754 floating-point numbers.
- Never compare floating-point numbers directly.
- Math object provides useful mathematical methods.
- Template literals use backticks and `${ }`.
- Template literals are heavily used in DOM manipulation, APIs, and React.

These four topics are among the most frequently used modern JavaScript features and appear regularly in React projects, frontend interviews, and real-world application development.